

Sequential Decision Making: Theory and Practice

Lecture 4: RL with a Simulator

Yuejie Chi

Yale Statistics & Data Science



Fall 2026

Outline

RL with a generative model

Model-based policy evaluation

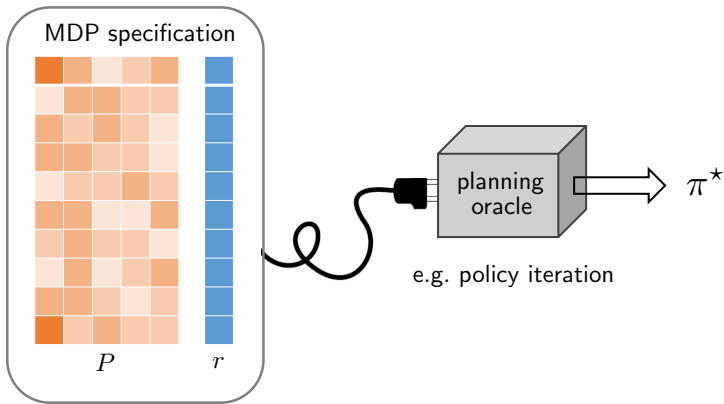
Model-based policy learning

Model-free policy evaluation: Monte Carlo

Model-free policy evaluation: TD learning

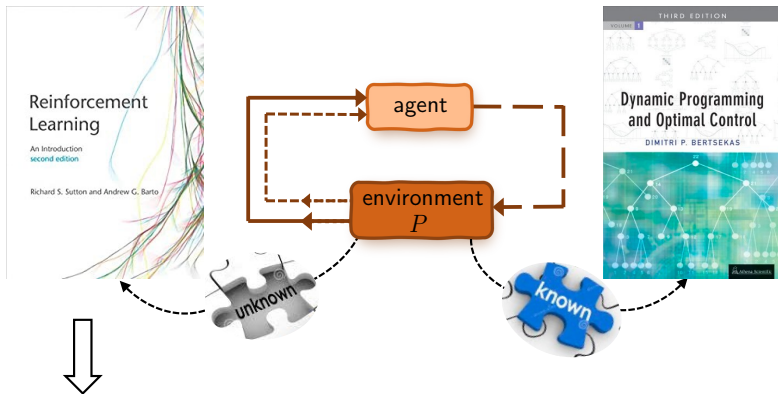
Model-free policy learning: Q-learning

Planning: when the MDP model is known



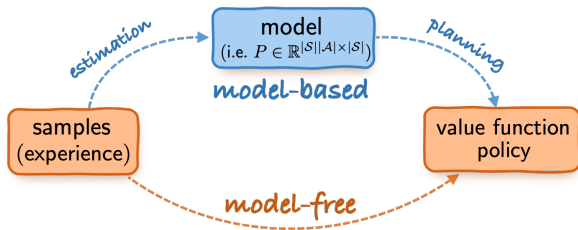
Planning: find the optimal policy π^* given MDP specification

Reinforcement learning (RL)



Learning: learn a desired policy from samples w/o model specification

Two approaches to RL



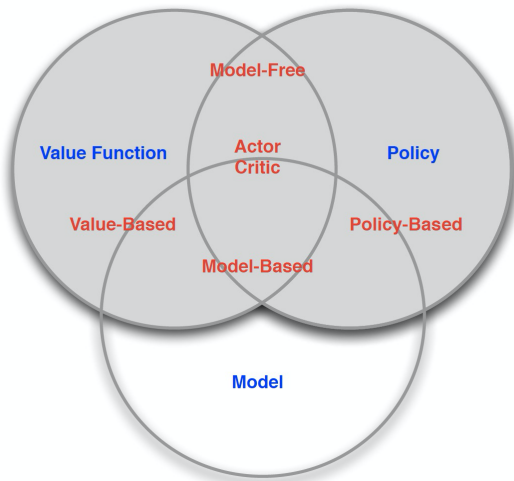
Model-based approach (“plug-in”)

1. build an empirical estimate $\hat{P}$ for P
2. planning based on empirical $\hat{P}$

Model-free approach

— learning w/o constructing model explicitly

A taxonomy of RL approaches



—Credit: David Silver's slide

RL with a generative model

Motivation: study sample efficiency

Collecting data samples might be expensive or time-consuming



clinical trials



autonomous driving



online ads

Understand and design of sample-efficient RL algorithms!

Data source in RL

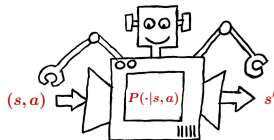
Exploration



offline RL



online RL



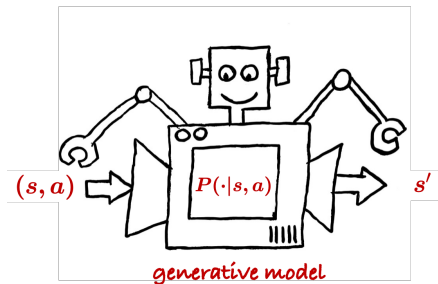
generative model

The capability of exploration increases from left to right.

This lecture: generative model / simulator

RL with a generative model / simulator

— [Kearns and Singh, 1999]

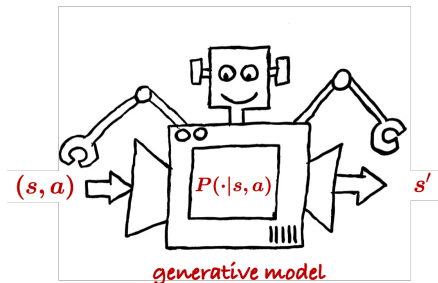


Protocol: for any state-action pair (s, a) , we can probe the simulator to output the next state s' .

We focus on the transition kernel and assume the reward is known or fixed, since the transition kernel captures the harder aspect of the problem.

RL with a generative model / simulator

— [Kearns and Singh, 1999]

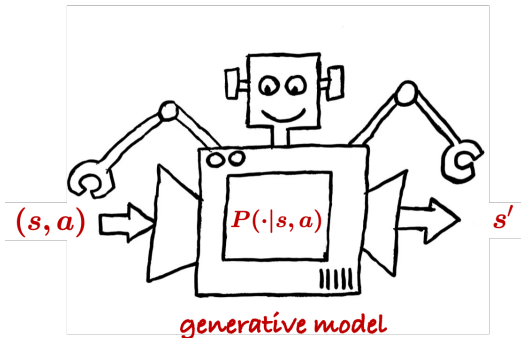


For each state-action pair (s, a) , collect N samples

$$\{(s, a, s'_{(i)})\}_{1 \leq i \leq N}$$

Question: How many samples are necessary and sufficient to solve the RL problem **without worrying about exploration?**

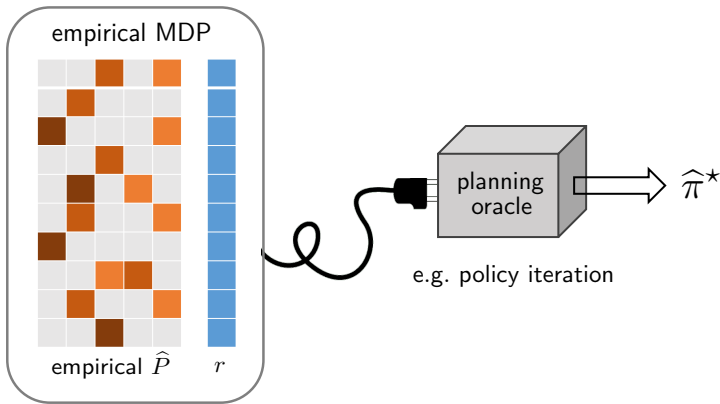
Model estimation under the generative model



For each (s, a) , collect N independent samples $\{(s, a, s'_{(i)})\}_{1 \leq i \leq N}$

Empirical estimates: estimate $\hat{P}(s'|s, a)$ by $\underbrace{\frac{1}{N} \sum_{i=1}^N \mathbb{1}\{s'_{(i)} = s'\}}_{\text{empirical frequency}}$

Model-based (plug-in) estimator



Run planning algorithms based on the *empirical* MDP

Questions

ℓ_∞ -sample complexity: how many samples are required for

1) evaluate an ε -accurate policy ?

$$\forall s: |\hat{V}^\pi(s) - V^\pi(s)| \leq \varepsilon$$

2) learn an ε -optimal policy ?

$$\forall s: V^{\hat{\pi}}(s) \geq V^*(s) - \varepsilon$$

Model-based policy evaluation

Minimax lower bound

Theorem 1 (minimax lower bound; [Pananjady and Wainwright, 2020])

Fix a policy π . For all $\varepsilon \in [0, \frac{1}{1-\gamma})$, there exists some MDP such that the total number of samples need to be **at least**

$$\tilde{\Omega} \left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3 \varepsilon^2} \right)$$

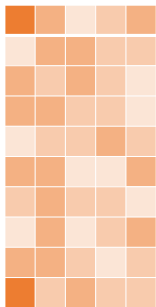
to achieve $\|\hat{Q} - Q^\pi\|_\infty \leq \varepsilon$, where $\hat{Q}$ is the output of any RL algorithm.

- Consider the relative accuracy ε_{rel} by setting $\varepsilon := \frac{\varepsilon_{\text{rel}}}{1-\gamma}$, the lower bound can be equivalently expressed as

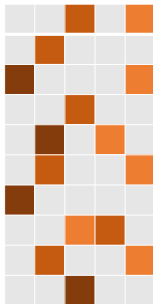
$$\tilde{\Omega} \left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma) \varepsilon_{\text{rel}}^2} \right).$$

- much smaller than the model dimension $|\mathcal{S}|^2 |\mathcal{A}|$ — hint at the possibility of evaluating the policy without estimating the model reliably!

Challenges in the sample-starved regime



truth: $P \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}|}$



empirical estimate: $\hat{P}$

- Can't recover P faithfully if sample size $\ll |\mathcal{S}|^2|\mathcal{A}|!$
- Can we trust our policy estimate when reliable model estimation is infeasible?

Recall: Bellman's consistency equation

- V^π / Q^π : value / action-value function under policy π

Bellman's consistency equation

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)]$$
$$Q^\pi(s, a) = \underbrace{r(s, a)}_{\text{immediate reward}} + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\underbrace{V^\pi(s')}_{\text{next state's value}} \right]$$

The value/Q function can be decomposed into two parts:

- immediate reward $\mathbb{E} [r(s, a)]$
- discounted value of at the successor state $\gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} V(s')$



Richard Bellman

Policy evaluation for state-action function

Matrix-vector representation:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a), a' \sim \pi(\cdot|s')} [Q^\pi(s', a')]$$

$$\Updownarrow$$

$$Q^\pi = r + \gamma P^\pi Q^\pi$$

$$\Updownarrow$$

$$Q^\pi = (I - \gamma P^\pi)^{-1} r$$

- Here, P^π is the state-action transition matrix induced by π , namely,

$$P^\pi(s', a'|s, a) = P(s'|s, a)\pi(a'|s').$$

Sample complexity for plug-in policy evaluation

Model-based plug-in estimate:

$$\hat{Q}^\pi = (I - \gamma \hat{P}^\pi)^{-1} r$$

Theorem 2 ([Li et al., 2024b])

Fix any policy π . For $0 < \varepsilon \leq \frac{1}{1-\gamma}$, the plug-in estimator $\hat{Q}^\pi$ obeys

$$\|\hat{Q}^\pi - Q^\pi\|_\infty \leq \varepsilon$$

with sample complexity at most

$$\tilde{O}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3\varepsilon^2}\right)$$

- Minimax optimal for all ε , improving upon prior analyses that only allow limited sample size [Azar et al., 2013, Pananjady and Wainwright, 2020].
- This is important in the most sample-limited regime, so we can say something as soon as $N \gtrsim \frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)}$.

Analysis: crude idea

- We'll demonstrate a crude version based on Hoeffding's inequality.

$$Q^\pi = (I - \gamma P^\pi)^{-1} r, \quad \hat{Q}^\pi = (I - \gamma \hat{P}^\pi)^{-1} r$$

Useful expansion:

$$\begin{aligned} \hat{Q}^\pi - Q^\pi &= (I - \gamma \hat{P}^\pi)^{-1} r - (I - \gamma P^\pi)^{-1} r \\ &= (I - \gamma \hat{P}^\pi)^{-1} \left((I - \gamma P^\pi) - (I - \gamma \hat{P}^\pi) \right) Q^\pi \\ &= \gamma (I - \gamma \hat{P}^\pi)^{-1} (\hat{P}^\pi - P^\pi) Q^\pi \\ &= \gamma (I - \gamma \hat{P}^\pi)^{-1} (\hat{P} - P) V^\pi. \end{aligned}$$

Analysis: Hoeffding's inequality

By Hoeffding's inequality and union bound, with probability at least $1 - \delta$,

$$\left\| (\hat{P} - P)V^\pi \right\|_\infty \leq \sqrt{\frac{2 \log(2|\mathcal{S}||\mathcal{A}|/\delta)}{N(1 - \gamma)^2}}.$$

Then, using $(I - \gamma \hat{P}^\pi)^{-1} = \sum_{i=0}^{\infty} \gamma^i (\hat{P}^\pi)^i$,

$$\begin{aligned} \left\| \gamma (I - \gamma \hat{P}^\pi)^{-1} (\hat{P} - P)V^\pi \right\|_\infty &\leq \gamma \sum_{i=0}^{\infty} \gamma^i \left\| (\hat{P}^\pi)^i (\hat{P} - P)V^\pi \right\|_\infty \\ &\leq \gamma \sum_{i=0}^{\infty} \gamma^i \left\| (\hat{P} - P)V^\pi \right\|_\infty \\ &\leq \frac{\gamma}{1 - \gamma} \left\| (\hat{P} - P)V^\pi \right\|_\infty \leq \gamma \sqrt{\frac{2 \log(2|\mathcal{S}||\mathcal{A}|/\delta)}{N(1 - \gamma)^4}}. \end{aligned}$$

This gives a fourth-order dependency with $\frac{1}{1 - \gamma}$.

Analysis: variance control + a peeling argument

- Better concentration with variance control: Bernstein's inequality
- Going beyond the 1st-order error expansion

$$\hat{Q}^\pi - Q^\pi = \gamma(I - \gamma P^\pi)^{-1}(\hat{P}^\pi - P^\pi)\hat{Q}^\pi$$

Instead: higher-order expansion $\longrightarrow$ tighter control

$$\begin{aligned}\hat{Q}^\pi - Q^\pi &= \gamma(I - \gamma P^\pi)^{-1}(\hat{P}^\pi - P^\pi)Q^\pi + \\ &\quad + \gamma^2 \left((I - \gamma P^\pi)^{-1}(\hat{P}^\pi - P^\pi) \right)^2 Q^\pi \\ &\quad + \gamma^3 \left((I - \gamma P^\pi)^{-1}(\hat{P}^\pi - P^\pi) \right)^3 Q^\pi \\ &\quad + \dots\end{aligned}$$

Model-based policy learning

Minimax lower bound

Theorem 3 (minimax lower bound; [Azar et al., 2013])

For all $\varepsilon \in [0, \frac{1}{1-\gamma})$, there exists some MDP such that the total number of samples need to be **at least**

$$\tilde{\Omega}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3\varepsilon^2}\right)$$

to achieve $\|\hat{Q} - Q^*\|_\infty \leq \varepsilon$, where $\hat{Q}$ is the output of any RL algorithm.

- holds for both value-based and policy-based algorithms.
- much smaller than the model dimension $|\mathcal{S}|^2|\mathcal{A}|$ — hint at the possibility of solving RL without estimating the model reliably!

Sample complexity for learning Q^*

Theorem 4 ([Azar et al., 2013])

For any $0 < \varepsilon \leq 1$, the optimal Q -function $\hat{Q}$ of the empirical MDP achieves

$$\|\hat{Q} - Q^*\|_\infty \leq \varepsilon$$

with sample complexity at most $\tilde{O}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3\varepsilon^2}\right)$.

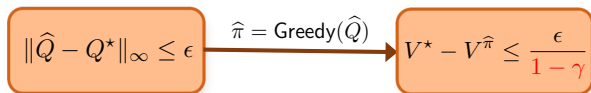
- matches with the minimax lower bound whenever $\varepsilon \in (0, 1]$.
- **Question:** Does it imply a near minimax-optimal policy $\hat{\pi}$?

From Q-function to policy

Lemma 5 ([Singh and Yee, 1994])

Let the greedy policy w.r.t. $\hat{Q}$ be $\hat{\pi}$, then

$$V^* - V^{\hat{\pi}} \leq \frac{2}{1-\gamma} \|\hat{Q} - Q^*\|_{\infty}.$$



This **error amplification** has consequences in sample complexities.

- To reach ϵ -optimality, the greedy policy of a minimax-optimal Q-function estimator needs

$$\tilde{O}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^5 \epsilon^2}\right)$$

samples invoking the above naive argument. Need refined arguments!

Theory of model-based policy learning

Theorem 6 ([Agarwal et al., 2020, Li et al., 2024b, Chen et al., 2026])

For any $0 < \varepsilon \leq \frac{1}{1-\gamma}$, the optimal policy $\hat{\pi}^*$ of the empirical MDP achieves

$$\|V^{\hat{\pi}^*} - V^*\|_{\infty} \leq \varepsilon$$

with sample complexity at most

$$\tilde{O}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3\varepsilon^2}\right)$$

- **Minimax lower bound:** $\tilde{\Omega}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3\varepsilon^2}\right)$ [Azar et al., 2013].

Model-based RL is nearly minimax optimal!

Notation and Bellman equation

- V^π : true value function under policy π
 - Bellman equation: $V^\pi = (I - \gamma P_\pi)^{-1} r$
- $\hat{V}^\pi$: estimate of value function under policy π
 - Bellman equation: $\hat{V}^\pi = (I - \gamma \hat{P}_\pi)^{-1} r$
- π^* : optimal policy w.r.t. true value function
- $\hat{\pi}^*$: optimal policy w.r.t. empirical value function
- $V^* := V^{\pi^*}$: optimal values under true models
- $\hat{V}^* := \hat{V}^{\hat{\pi}^*}$: optimal values under empirical models

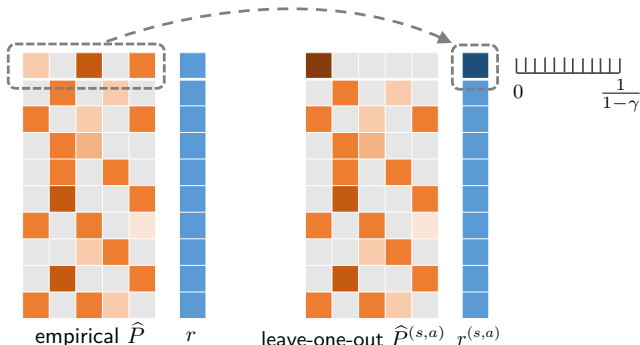
Proof ideas

Elementary decomposition:

$$\begin{aligned} V^* - V^{\widehat{\pi}^*} &= (V^* - \widehat{V}^{\pi^*}) + (\widehat{V}^{\pi^*} - \widehat{V}^{\widehat{\pi}^*}) + (\widehat{V}^{\widehat{\pi}^*} - V^{\widehat{\pi}^*}) \\ &\leq (V^{\pi^*} - \widehat{V}^{\pi^*}) + \textcolor{red}{0} + (\widehat{V}^{\widehat{\pi}^*} - V^{\widehat{\pi}^*}) \end{aligned}$$

- **Step 1:** control $V^{\pi} - \widehat{V}^{\pi}$ for a fixed π
(**Bernstein inequality** + **high-order decomposition**)
- **Step 2:** extend it to control $\widehat{V}^{\widehat{\pi}^*} - V^{\widehat{\pi}^*}$ ($\widehat{\pi}^*$ depends on samples)
(**decouple statistical dependency via leave-one-out analysis**)

A glimpse of the leave-one-out analysis

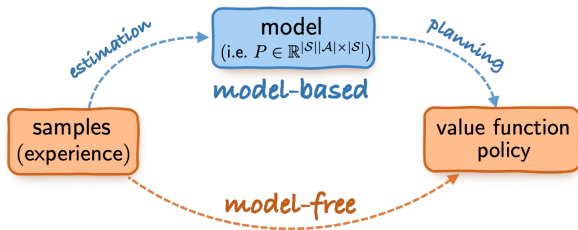


1. embed all randomness from $\hat{P}_{s,a}$ into a single scalar (i.e. $r_{s,a}^{(s,a)}$)
2. build an ϵ -net for this scalar
3. harnessing the statistical independence in leave-one-out MDPs

$$(\hat{P}_{s,a} - P_{s,a})\hat{V}^* \approx (\hat{P}_{s,a} - P_{s,a})\hat{V}^{\text{loo}}$$

Property of $\hat{V}^{\text{loo}}$: independent of $\hat{P}_{s,a}$ while being close to $\hat{V}^*$.

Two approaches to RL



Model-based approach (“plug-in”)

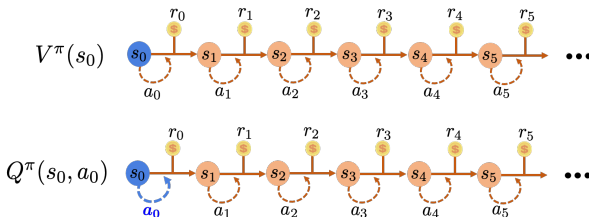
1. build an empirical estimate $\hat{P}$ for P
2. planning based on empirical $\hat{P}$

Model-free approach

— learning w/o constructing model explicitly

Model-free policy evaluation

Monte Carlo policy evaluation



Monte Carlo (MC) learns directly from experience by replacing the expectation by empirical means.

- Sample trajectories according to π
- Calculate the value using empirical means.

$$V^\pi(s) := \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right], \quad Q^\pi(s, a) := \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

Monte Carlo policy evaluation

Consider a trajectory rolled out by following policy π :

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots,$$

The **return** or **reward-to-go** from time t is

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} \dots$$

- $V^\pi(s) = \mathbb{E}[G_t | s_t = s];$

Idea: to evaluate state s , average the reward-to-gos from time-steps that visit state s over many trajectories.

$$V(s) \approx \frac{\sum_{t:s_t=s} G_t}{\sum_{t:s_t=s} 1}$$

First-visit versus Every-visit

First-visit Monte Carlo:

For each episode, at the **first** time-step t that state s is visited in an episode.

- Increase the counter $N(s) \leftarrow N(s) + 1$
- Increase the total return $S(s) \leftarrow S(s) + G_t$
- Value is estimated by mean return: $V(s) = S(s)/N(s)$

One return per visited episode

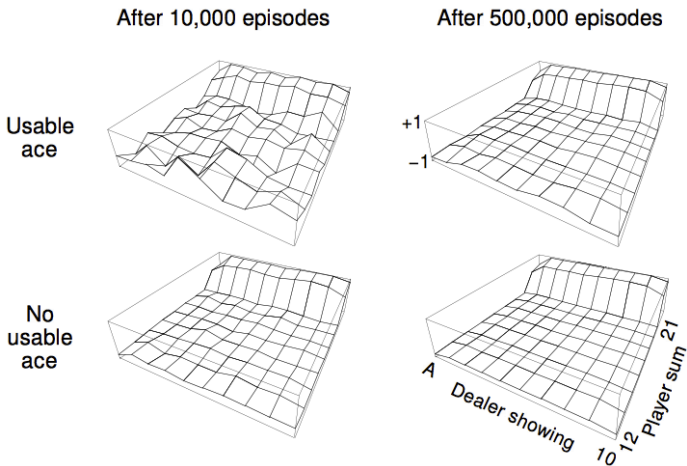
Every-visit Monte Carlo:

For each episode, at the **every** time-step t that state s is visited in an episode.

- Increase the counter $N(s) \leftarrow N(s) + 1$
- Increase the total return $S(s) \leftarrow S(s) + G_t$
- Value is estimated by mean return: $V(s) = S(s)/N(s)$

Several, correlated returns per episode

Example: blackjack Monte-Carlo value estimation



Policy: stick if sum of cards ≥ 20 , otherwise twist.

Incremental Monte Carlo update

The Monte-Carlo value update can be done in an incremental manner to facilitate implementation.

$$\begin{aligned} N(s_t) &\leftarrow N(s_t) + 1 \\ V(s_t) &\leftarrow V(s_t) + \underbrace{\frac{1}{N(s_t)} (G_t - V(s_t))}_{\text{incremental update}} \end{aligned}$$

The value $V(s_t)$ is updated towards the actual return G_t .

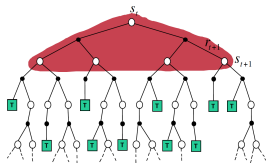
This motivates a more general scheme, which is beneficial specially in non-stationary problems, that one simply does

$$V(s_t) \leftarrow V(s_t) + \alpha (G_t - V(s_t)),$$

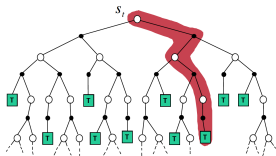
where α is the learning rate to enable more flexible trade-off between past and future (e.g., forgetting faster when $\alpha > \frac{1}{N(s_t)}$).

Dynamic programming versus Monte Carlo

Monte Carlo estimates returns directly, without a Bellman backup.



Dynamic programming
bootstrapping



Monte Carlo
sampling

Caveat of Monte Carlo methods:

- Must wait until the episode ends to calculate the full return.
- For continuing tasks, use finite rollouts with a truncation error.
- Generally incurs a high variance, but consistent under mild conditions.

Temporal difference (TD) learning

Temporal difference (TD) learning

"If one had to identify one idea as central and novel to RL, it would undoubtedly be TD learning."



Richard Sutton

Temporal difference (TD) learning

- combines dynamic programming and Monte Carlo, by bootstrapping and sampling simultaneously
- learns from incomplete episodes, and does not require the episode to terminate
- "updates a guess towards a guess"

TD learning for value evaluation

- In Monte Carlo, updating the value towards the return:

$$V(s_t) \leftarrow V(s_t) + \alpha (G_t - V(s_t))$$

- Instead, TD updates $V(s_t)$ towards estimated return $r_t + \gamma V(s_{t+1})$

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{\left(\underbrace{r_t + \gamma V(s_{t+1})}_{\text{TD target}} - V(s_t) \right)}_{\text{TD error}}$$

- TD target $r_t + \gamma V(s_{t+1})$: sampling + bootstrapping
- TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

TD-learning as stochastic approximation

Stochastic approximation [Robbins and Monro, 1951] for solving Bellman equation

$$V = \mathcal{T}^\pi(V),$$

where the Bellman operator $\mathcal{T}^\pi : \mathbb{R}^{|\mathcal{S}|} \mapsto \mathbb{R}^{|\mathcal{S}|}$ is defined as

$$\forall V \in \mathbb{R}^{|\mathcal{S}|} : \quad \mathcal{T}^\pi(V) = r_\pi + \gamma P_\pi V.$$

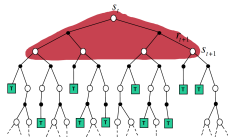
- Access a stochastic realization of $\mathcal{T}^\pi(V)$:

$$\mathcal{T}_t^\pi(V)(s_t) = r_t + \gamma V(s_{t+1})$$

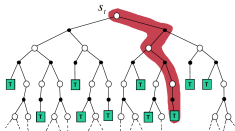
- Update $V(s_t)$ by a weighted combination of old and new:

$$\begin{aligned} V(s_t) &\leftarrow (1 - \alpha)V(s_t) + \alpha \mathcal{T}_t^\pi(V)(s_t) \\ &= V(s_t) + \underbrace{\alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right]}_{\text{temporal difference}}, \quad t \geq 0 \end{aligned}$$

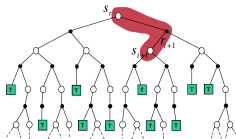
DP versus MC versus TD



Dynamic programming
bootstrapping



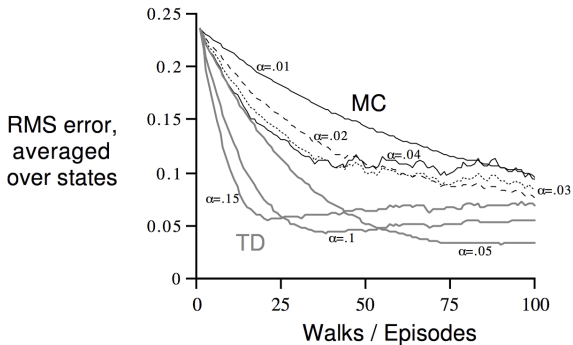
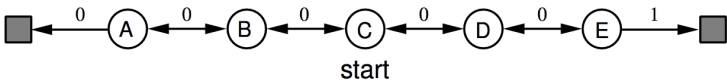
Monte Carlo
sampling



TD learning
bootstrapping+sampling

- TD often has lower target variance, but bootstraps from an estimate.
- TD learns on-the-fly because of bootstrapping.

Example: random walk



Sample complexity under the generative model

Synchronous TD-learning:

$$V_{t+1}(s) = \underbrace{(1 - \eta_t)V_t(s) + \eta_t \mathcal{T}_t(V_t)(s)}_{\text{draw the transition } (s,s') \text{ for all } (s)}, \quad t \geq 0$$

where $\mathcal{T}_t(V)(s) = r(s) + \gamma V(s')$.

Theorem 7 ([Li et al., 2024a, Chen et al., 2026])

For any $0 < \varepsilon \leq \frac{1}{1-\gamma}$, TD-learning yields

$$\|\hat{V} - V^*\|_\infty \leq \varepsilon$$

with sample complexity *at most*

$$\tilde{O}\left(\frac{|\mathcal{S}|}{(1-\gamma)^3 \varepsilon^2}\right).$$

- Near minimax-optimal without the need of averaging or variance reduction.

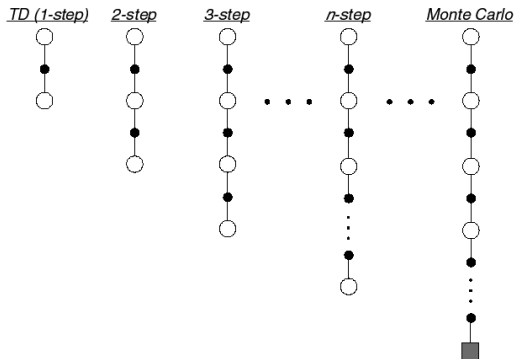
n -step TD

Let the TD target look n steps into the future

$$V^\pi(s) = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) | s_t = s] \quad (\text{one-step bootstrap})$$

$$= \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 V^\pi(s_{t+2}) | s_t = s] \quad (\text{two-step bootstrap})$$

$$= \dots$$



n -step TD

The n -step return:

$$G_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j r_{t+j} + \gamma^n V(s_{t+n})$$

- $n = 1$: TD target
- Full episode (or $n \rightarrow \infty$ with discounting): MC target

The n -step TD learning:

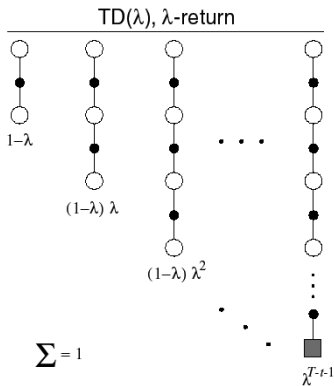
$$V(s_t) \leftarrow V(s_t) + \underbrace{\alpha (G_t^{(n)} - V(s_t))}_{\text{TD error}}$$

- Mix-and-match: combine information over different n as the TD target, e.g. using

$$\frac{1}{2} G_t^{(2)} + \frac{1}{2} G_t^{(3)}.$$

From n -step TD to $\text{TD}(\lambda)$

Can we efficiently combine information from all time-steps?



The λ -**return** G_t^λ combines all n -step returns using weight $(1 - \lambda)\lambda^{n-1}$:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

where $\lambda \in [0, 1]$.

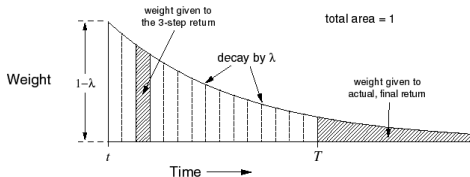
The forward-view $\text{TD}(\lambda)$:

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{(G_t^\lambda - V(s_t))}_{\text{TD error}}$$

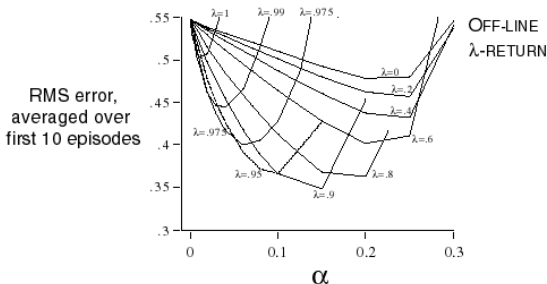
Update towards the λ -return.

Forward-view TD(λ)

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$



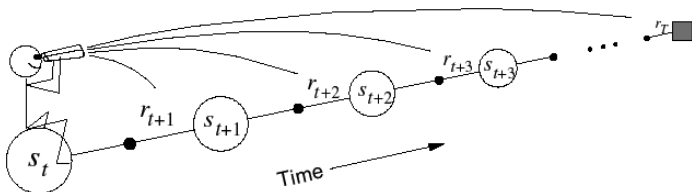
Example: forward-view TD(λ) on random walk



From forward-view TD(λ) to backward-view TD(λ)

Forward-view TD(λ):

- Like MC, requires the episode to terminate to compute G_t^λ .

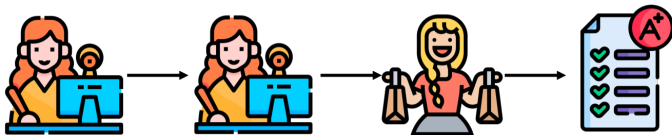


Question

Can we have TD(λ) run on-the-fly?

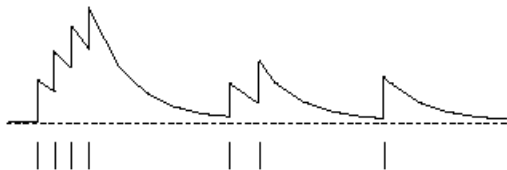
Eligibility traces

Credit assignment: most frequent or most recent



Eligibility traces combine both heuristics:

$$E_t(s) = \gamma \lambda E_{t-1}(s) + \mathbb{I}(s_t = s), \quad \text{with } E_{-1}(s) = 0$$



accumulating eligibility trace

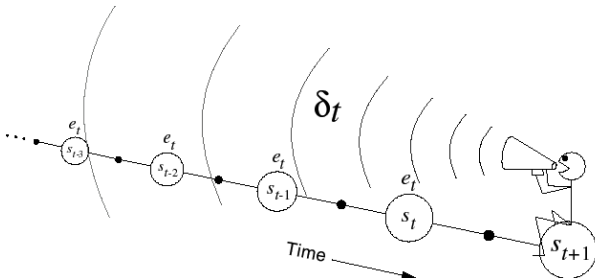
times of visits to a state

Backward-view TD(λ)

- Keep an eligibility trace for every state s
- Update value $V(s)$ for **every state** s , in proportion to TD-error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ and eligibility trace $E_t(s)$:

$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s)$$

When $\lambda = 0$, $E_t(s) = \mathbb{I}(s_t = s)$, and it reduces to TD(0), the basic TD.



Equivalence of forward/backward-view TD(λ)

- Episode ends at s_T , with $V(s_T) = 0$. Hold V fixed.

Theorem 8

For a fixed value estimate, the accumulated updates agree:

$$\underbrace{\sum_{t=0}^{T-1} \alpha \delta_t E_t(s)}_{\text{backward updates}} = \underbrace{\sum_{t=0}^{T-1} \alpha (G_t^\lambda - V(s_t)) \mathbb{I}(s_t = s)}_{\text{forward updates}}$$

- Forward view provides theory
- Backward view provides mechanism

Forward/backward-view TD(λ)

Consider an episode where s is visited once at time-step k .

- TD(λ) eligibility trace discounts time since visit,

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbb{I}(s_t = s) = \begin{cases} 0, & t < k \\ (\gamma\lambda)^{t-k}, & t \geq k \end{cases}$$

- Backward TD(λ) accumulates errors *online*:

$$\sum_{t=0}^{T-1} \alpha \delta_t E_t(s) = \alpha \sum_{t=k}^{T-1} (\gamma\lambda)^{t-k} \delta_t = \alpha (G_k^\lambda - V(s_k))$$

- By end of episode it accumulates total error for λ -return

Telescoping in TD(λ)

For fixed V and $0 \leq \lambda < 1$, TD errors telescope:

$$\begin{aligned} \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots \\ &= \textcolor{red}{r}_t + \gamma V(s_{t+1}) - V(s_t) \\ &\quad + (\gamma\lambda)r_{t+1} + \gamma(\gamma\lambda)V(s_{t+2}) - \gamma\lambda V(s_{t+1}) \\ &\quad + (\gamma\lambda)^2r_{t+2} + \gamma(\gamma\lambda)^2V(s_{t+3}) - (\gamma\lambda)^2V(s_{t+2}) + \dots \\ &= -V(s_t) + (1-\lambda)\lambda^0(\textcolor{red}{r}_t + \gamma V(s_{t+1})) \\ &\quad + (1-\lambda)\lambda^1(\textcolor{red}{r}_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})) \\ &\quad + (1-\lambda)\lambda^2(\textcolor{red}{r}_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})) + \dots \\ &= G_t^\lambda - V(s_t) \end{aligned}$$

where

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \left[\sum_{j=0}^{n-1} \gamma^j r_{t+j} + \gamma^n V(s_{t+n}) \right].$$

Model-free policy learning: Q-learning

Q-learning: a classical model-free algorithm



Chris Watkins



Peter Dayan

Stochastic approximation for solving the **Bellman equation**

Robbins & Monro, 1951

$$Q = \mathcal{T}(Q)$$

where

$$\mathcal{T}(Q)(s, a) := \underbrace{r(s, a)}_{\text{immediate reward}} + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\underbrace{\max_{a' \in \mathcal{A}} Q(s', a')}_{\text{next state's value}} \right].$$

Q-learning: a classical model-free algorithm



Chris Watkins



Peter Dayan

Q-learning [Watkins and Dayan, 1992] proceeds as

$$\begin{aligned} Q_{t+1}(s, a) &= \underbrace{(1 - \eta_t)Q_t(s, a) + \eta_t \mathcal{T}_t(Q_t)(s, a)}_{\text{draw the transition } (s, a, s') \text{ for all } (s, a)}, \quad t \geq 0 \\ &= Q_t(s, a) + \eta_t \left(r(s, a) + \gamma \max_{a'} Q_t(s', a') - Q_t(s, a) \right) \end{aligned}$$

$$\mathcal{T}_t(Q)(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a')$$

$$\mathcal{T}(Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\max_{a'} Q(s', a') \right]$$

Asymptotic convergence

Theorem 9 ([Watkins and Dayan, 1992])

Q-learning converges to the optimal Q-function Q^ asymptotically with probability 1 as long as*

$$\sum_{t=1}^{\infty} \eta_t = \infty, \quad \sum_{t=1}^{\infty} \eta_t^2 < \infty.$$

- The first condition asks the learning rates to be not too small, while the second condition ensures that they are not too large.
- Many choices of learning rates satisfy this assumption.

What about the finite-time convergence rate of Q-learning?

A note on the learning rates

Observation: the learning rate schedule $\eta_t = \frac{1}{t}$ leads to a sample complexity that scales **exponentially** in $\frac{1}{1-\gamma}$.

- Consider the following MDP with a single state $s = 1$, a single action $a = 1$, and $r(1, 1) = 1$, $P(1|1, 1) = 1$. Hence,

$$Q^*(1, 1) = \frac{1}{1 - \gamma}.$$

- The update rule of Q-learning with learning rate $\eta_t = \frac{1}{t}$ gives

$$\begin{aligned} Q_t(1, 1) &= \left(1 - \frac{1}{t}\right) Q_{t-1}(1, 1) + \frac{1}{t} (1 + \gamma Q_{t-1}(1, 1)) \\ &= \left(1 - \frac{1 - \gamma}{t}\right) Q_{t-1}(1, 1) + \frac{1}{t}, \end{aligned}$$

A note on the learning rates - continued

- From simple recursive relations, one can easily check that: when $\gamma \rightarrow 1$ and t is not too large, one has

$$\begin{aligned} Q_t - Q^* &= \prod_{i=1}^t \left[1 - \frac{1-\gamma}{i} \right] \cdot [Q_0 - Q^*] \\ &\approx \left[1 - \sum_{i=1}^t \frac{1-\gamma}{i} \right] \cdot [Q_0 - Q^*] \\ &\approx [1 - (1-\gamma) \log t] \cdot [Q_0 - Q^*]. \end{aligned}$$

This essentially implies that one needs to have $t \gtrsim 2^{O(\frac{1}{1-\gamma})}$ iterations to achieve $|Q_t - Q^*| < \frac{1}{2}|Q_0 - Q^*|$.

Consequently, the **rescaled** linear learning rates or constant learning rates provide better alternatives.

Sample complexity of Q-learning

Theorem 10 ([Li et al., 2024a, Chen et al., 2026])

For any $0 < \varepsilon \leq \frac{1}{1-\gamma}$, Q-learning yields

$$\|\hat{Q} - Q^*\|_\infty \leq \varepsilon$$

with sample complexity *at most*

$$\tilde{O}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^4\varepsilon^2}\right).$$

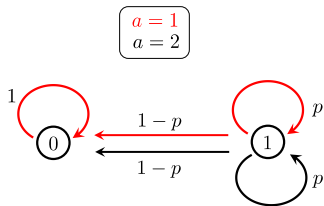
- Allows both constant and rescaled linear learning rate:

$$\frac{1}{1 + \frac{c_1(1-\gamma)T}{\log^2 T}} \leq \eta_t \leq \frac{1}{1 + \frac{c_2(1-\gamma)t}{\log^2 T}}$$

A curious numerical example

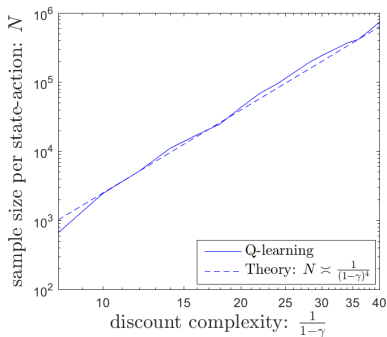
Numerical evidence: $\frac{|S||\mathcal{A}|}{(1-\gamma)^4 \epsilon^2}$ samples seem necessary ...

— observed in [Wainwright, 2019]



$$p = \frac{4\gamma - 1}{3\gamma}$$

$$r(0, 1) = 0, \quad r(1, 1) = r(1, 2) = 1$$



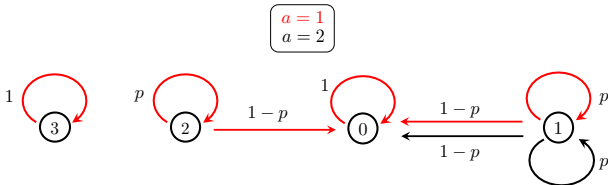
Q-learning is not minimax optimal

Theorem 11 ([Li et al., 2024a])

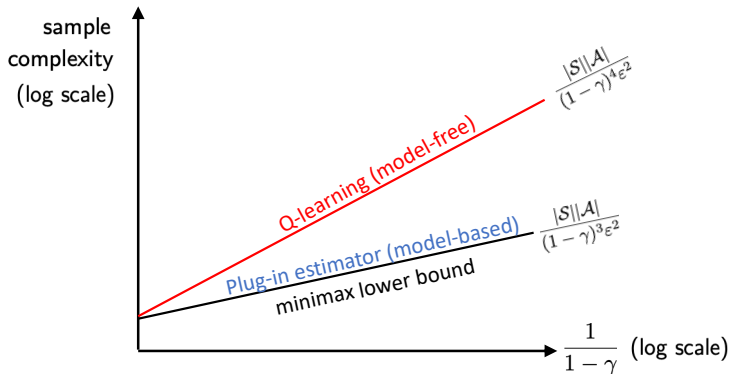
For any $0 < \varepsilon \leq 1$, there exist an MDP such that to achieve $\|\hat{Q} - Q^*\|_\infty \leq \varepsilon$, Q-learning needs **at least** a sample complexity of

$$\tilde{\Omega}\left(\frac{|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^4\varepsilon^2}\right).$$

- Tight **algorithm-dependent** lower bound
- Holds for both constant and rescaled linear learning rates



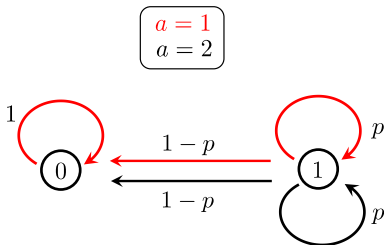
Where we stand now



Plug-in estimators are minimax-optimal, while Q-learning is not.

Why is Q-learning sub-optimal?

Over-estimation of Q-functions [Thrun and Schwartz, 1993, Hasselt, 2010]:



- $\max_{a \in \mathcal{A}} \mathbb{E}X(a)$ is over-estimated (high positive bias) when $\mathbb{E}X(a)$ is replaced by its empirical estimates using a small sample size

$$\forall a \in \mathcal{A} : \quad \max_{b \in \mathcal{A}} X(b) \geq X(a)$$

$$\implies \quad \mathbb{E}[\max_{a \in \mathcal{A}} X(a)] \geq \max_{a \in \mathcal{A}} \mathbb{E}[X(a)]$$

Why is Q-learning sub-optimal?

The over-estimation of Q-functions often gets **worse** with a large number of actions [Van Hasselt et al., 2016].

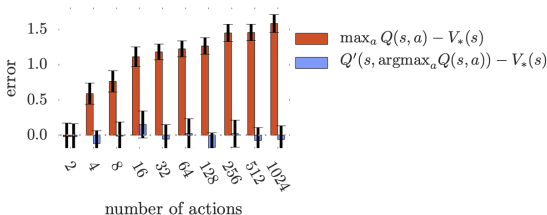


Figure 1: The orange bars show the bias in a single Q-learning update when the action values are $Q(s, a) = V_*(s) + \epsilon_a$ and the errors $\{\epsilon_a\}_{a=1}^m$ are independent standard normal random variables. The second set of action values Q' , used for the blue bars, was generated identically and independently. All bars are the average of 100 repetitions.

Double Q-learning

To mitigate the impact of over-estimation, [Hasselt, 2010] proposed **double Q-learning**, which uses two Q-estimates and updates one of them randomly at each round:

$$Q^1(s, a) = (1 - \eta_t)Q^1(s, a) + \eta_t \left(r(s, a) + \gamma Q^2(s', \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q^1(s', a)) \right),$$

or

$$Q^2(s, a) = (1 - \eta_t)Q^2(s, a) + \eta_t \left(r(s, a) + \gamma Q^1(s', \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q^2(s', a)) \right).$$

- Decouple the randomness in value updates and action selection.
- Empirically very successful when integrated with deep RL [Van Hasselt et al., 2016].

References I

-  Agarwal, A., Kakade, S., and Yang, L. F. (2020).
Model-based reinforcement learning with a generative model is minimax optimal.
Conference on Learning Theory, pages 67–83.
-  Azar, M. G., Munos, R., and Kappen, H. J. (2013).
Minimax PAC bounds on the sample complexity of reinforcement learning with a generative model.
Machine learning, 91(3):325–349.
-  Chen, Y., Chi, Y., Fan, J., Li, G., Wei, Y., and Yan, Y. (2026).
Statistical foundations of reinforcement learning: A non-asymptotic perspective.
Monograph. Under review at *Foundations and Trends in Statistics*.
-  Hasselt, H. (2010).
Double Q-learning.
Advances in neural information processing systems, 23:2613–2621.
-  Kearns, M. J. and Singh, S. P. (1999).
Finite-sample convergence rates for Q-learning and indirect algorithms.
In *Advances in neural information processing systems*, pages 996–1002.
-  Li, G., Cai, C., Chen, Y., Wei, Y., and Chi, Y. (2024a).
Is Q-learning minimax optimal? a tight sample complexity analysis.
Operations Research, 72(1):222–236.

References II



Li, G., Wei, Y., Chi, Y., and Chen, Y. (2024b).

Breaking the sample size barrier in model-based reinforcement learning with a generative model.
Operations Research, 72(1):203–221.



Pananjady, A. and Wainwright, M. J. (2020).

Instance-dependent ℓ_∞ -bounds for policy evaluation in tabular reinforcement learning.
IEEE Transactions on Information Theory, 67(1):566–585.



Robbins, H. and Monro, S. (1951).

A stochastic approximation method.
The annals of mathematical statistics, pages 400–407.



Singh, S. P. and Yee, R. C. (1994).

An upper bound on the loss from approximate optimal-value functions.
Machine Learning, 16:227–233.



Thrun, S. and Schwartz, A. (1993).

Issues in using function approximation for reinforcement learning.
In *Proceedings of the Fourth Connectionist Models Summer School*, volume 255, page 263.
Hillsdale, NJ.



Van Hasselt, H., Guez, A., and Silver, D. (2016).

Deep reinforcement learning with double Q-learning.
In *Proceedings of the AAAI conference on artificial intelligence*, volume 30.

References III



Wainwright, M. J. (2019).

Stochastic approximation with cone-contractive operators: Sharp ℓ_∞ -bounds for Q-learning.
arXiv preprint arXiv:1905.06265.



Watkins, C. J. and Dayan, P. (1992).

Q-learning.
Machine learning, 8(3-4):279–292.